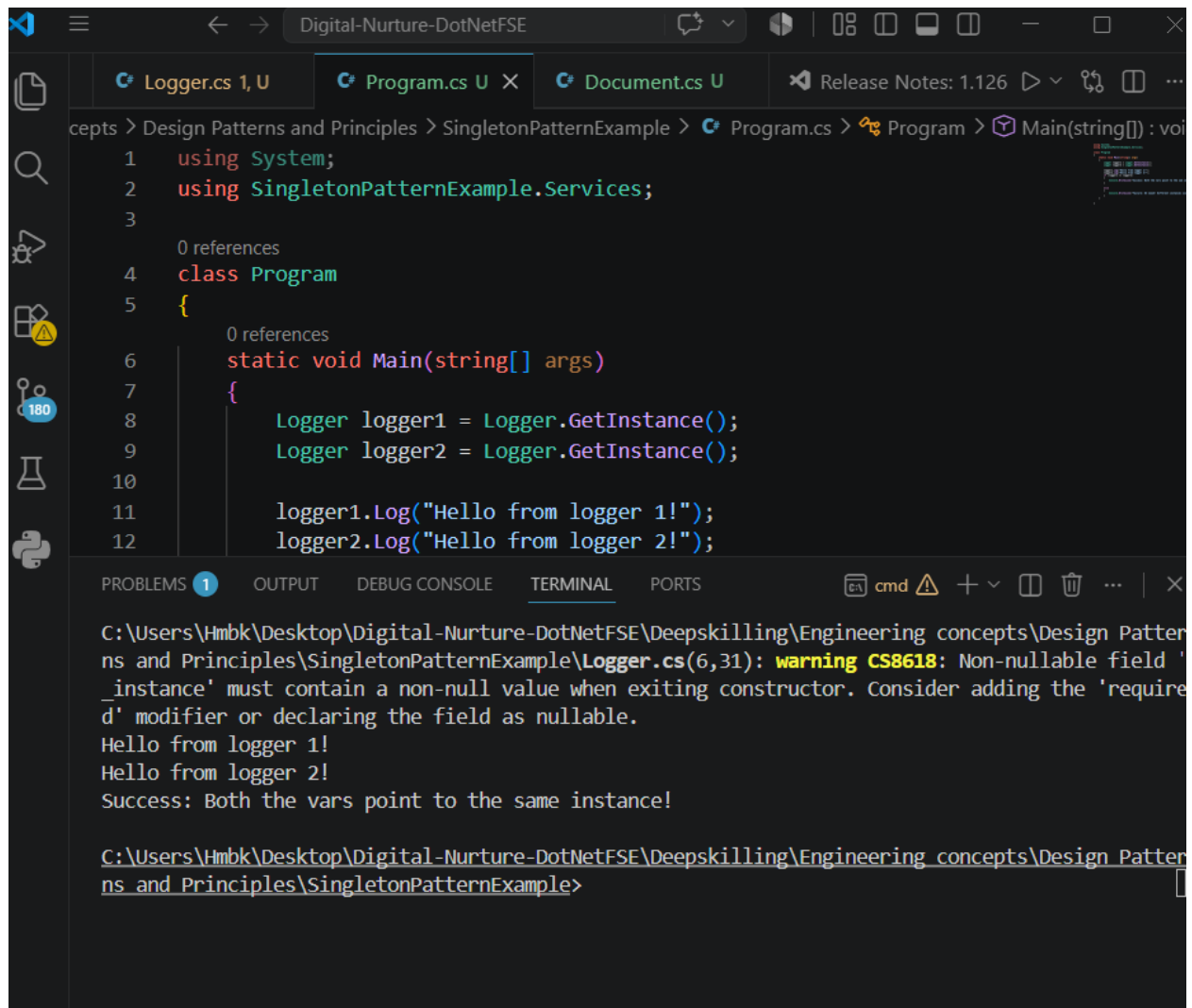
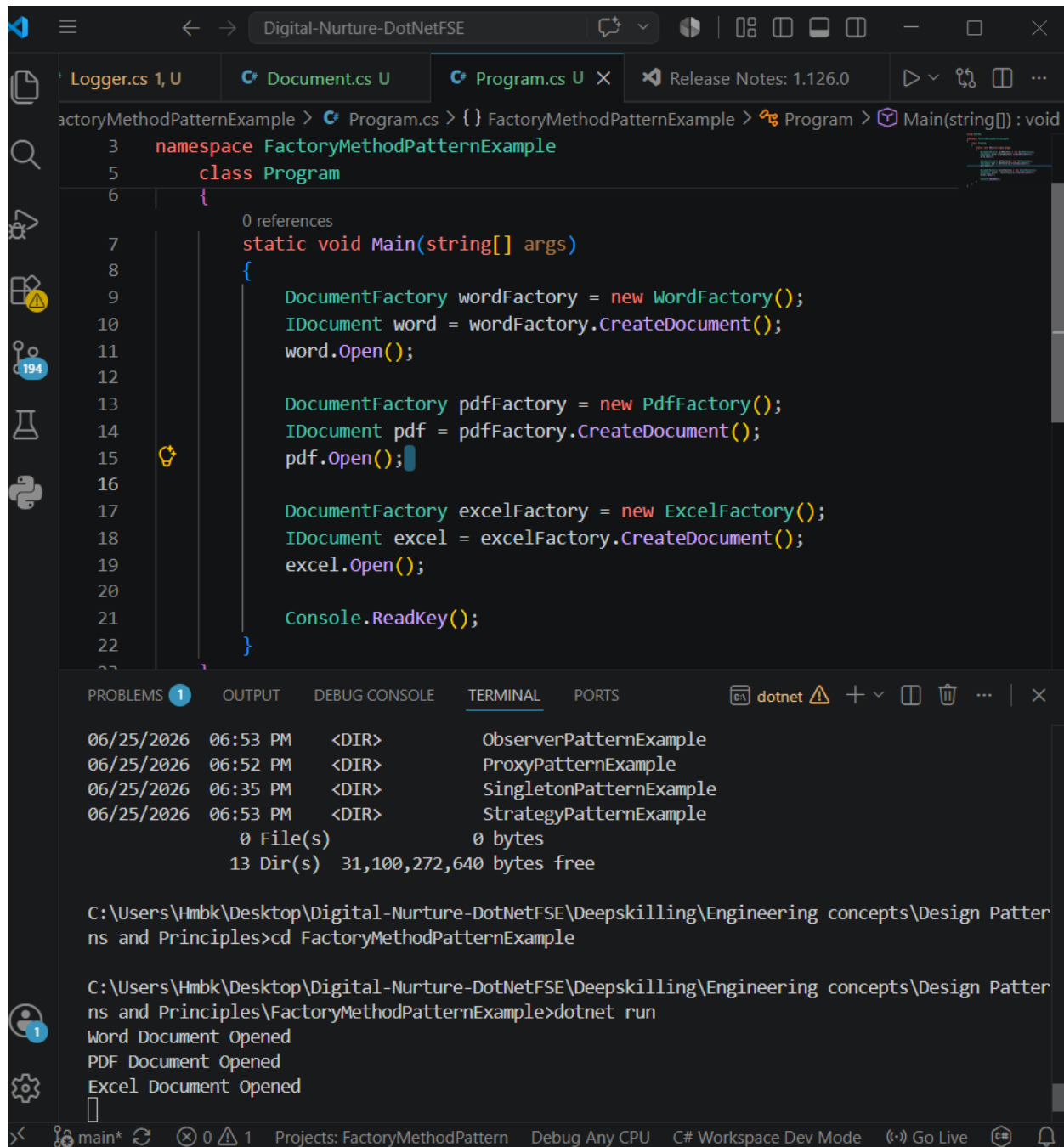
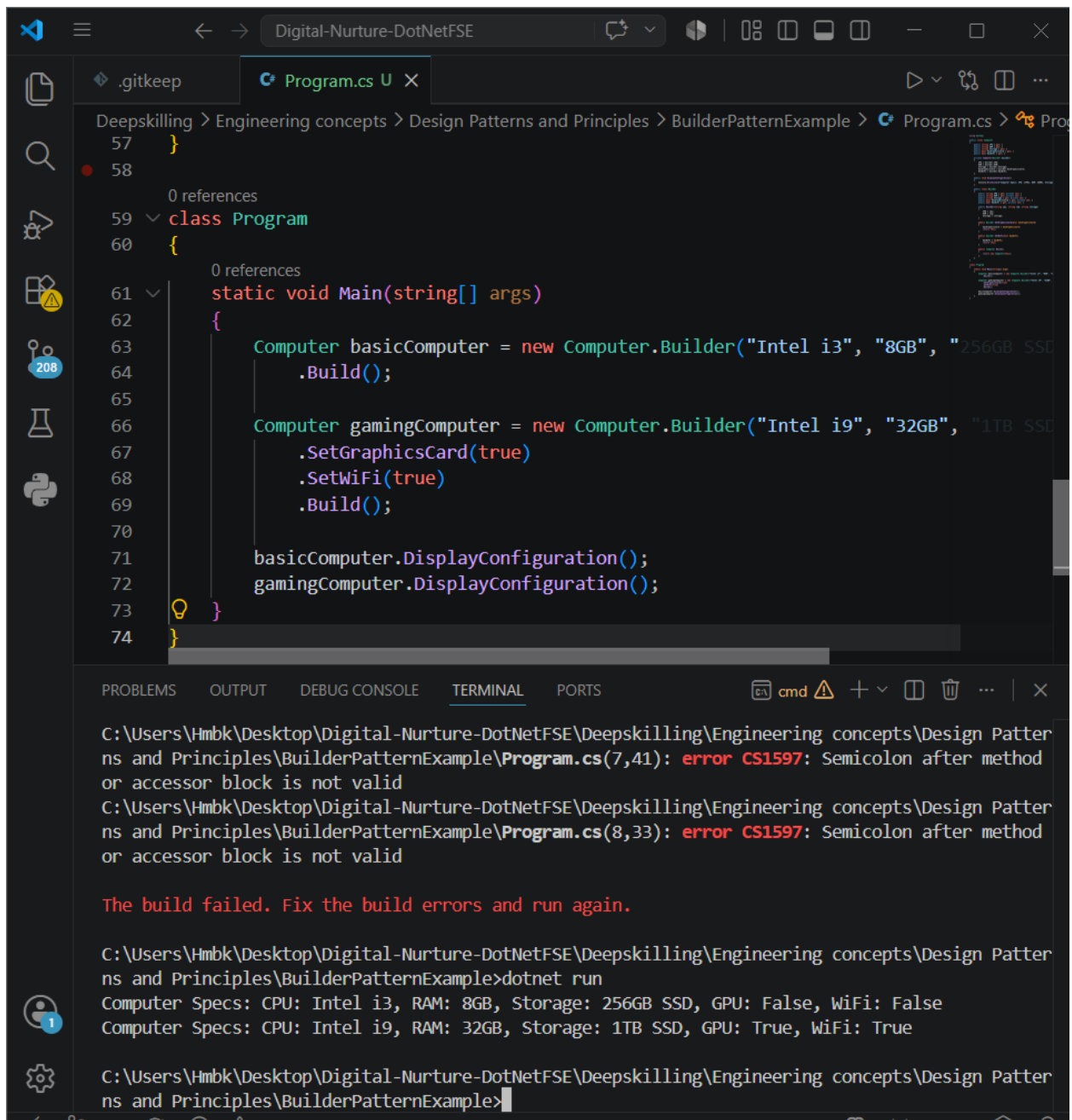








Visual Studio Code interface showing a C# program implementing the Adapter Pattern. The code is in `Program.cs` under the project `AdapterPatternExample`.

```
51 }  
52 0 references  
53 class Program  
54 {  
55     0 references  
56     static void Main(string[] args)  
57     {  
58         PhonePeGateway phonePe = new PhonePeGateway();  
59         IPaymentProcessor phonePeProcessor = new PhonePeAdapter(phonePe);  
60  
61         GooglePayGateway gpay = new GooglePayGateway();  
62         IPaymentProcessor gpayProcessor = new GooglePayAdapter(gpay);  
63  
64         phonePeProcessor.ProcessPayment(150.00);  
65         gpayProcessor.ProcessPayment(500.00);  
66     }  
67 }
```

The terminal window shows the following output and commands:

```
Computer Specs: CPU: Intel i3, RAM: 8GB, Storage: 256GB SSD, GPU: False, WiFi: False  
Computer Specs: CPU: Intel i9, RAM: 32GB, Storage: 1TB SSD, GPU: True, WiFi: True  
  
C:\Users\Hmbk\Desktop\Digital-Nurture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles\BuilderPatternExample>cd ..  
  
C:\Users\Hmbk\Desktop\Digital-Nurture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles>cd AdapterPatternExample  
  
C:\Users\Hmbk\Desktop\Digital-Nurture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles\AdapterPatternExample>dotnet run  
Processing $150 via PhonePe UPI  
Processing $500 via Google Pay.  
  
C:\Users\Hmbk\Desktop\Digital-Nurture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles\AdapterPatternExample>
```

Visual Studio interface showing the implementation of the Decorator Pattern in C#.

Code Editor:

```
3 namespace DecoratorPatternExample
33 public class SMSNotifierDecorator : NotifierDecorator
44 {
45
46     2 references
47     public class SlackNotifierDecorator : NotifierDecorator
48     {
49         1 reference
50         public SlackNotifierDecorator(INotifier notifier) : base(notifier)
51     {
52         6 references
53         public override void Send()
54         {
55             base.Send();
56             Console.WriteLine("Sending notification via Slack");
57         }
58     }
```

Terminal:

```
ns and Principles\AdapterPatternExample>cd
C:\Users\Hmbk\Desktop\Digital-Nuture-DotNetFSE\Deepskilling\Engineering concepts\Design Pattern
ns and Principles\AdapterPatternExample

C:\Users\Hmbk\Desktop\Digital-Nuture-DotNetFSE\Deepskilling\Engineering concepts\Design Pattern
ns and Principles\AdapterPatternExample>cd ..

C:\Users\Hmbk\Desktop\Digital-Nuture-DotNetFSE\Deepskilling\Engineering concepts\Design Pattern
ns and Principles>cd DecoratorPatternExample

C:\Users\Hmbk\Desktop\Digital-Nuture-DotNetFSE\Deepskilling\Engineering concepts\Design Pattern
ns and Principles\DecoratorPatternExample>dotnet run
Sending notification via Email
Sending notification via SMS
Sending notification via Slack
```

Status Bar: main* 0 0 Projects: DecoratorPattern Debug Any CPU C# Workspace Dev Mode (Go Live)

VS Code Explorer: Digital-Nurture-DotNetFSE

Program.cs ...\DecoratorPatternExample U Program.cs ...\ProxyPatternExample 1, U X

Deepskilling > Engineering concepts > Design Patterns and Principles > ProxyPatternExample > Program.cs > {} Prox

```
3 namespace ProxyPatternExample
10 class RealImage : IImage
    1 reference
14 public RealImage(string fileName)
15 {
16     this.fileName = fileName;
17     LoadFromServer();
18 }
19
    1 reference
20 private void LoadFromServer()
21 {
22     Console.WriteLine("Loading " + fileName + " from remote server...");
23 }
24
    4 references
25 public void Display()
26 {
27     Console.WriteLine("Displaying " + fileName);
28 }
29 }
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

C:\Users\Hmbk\Desktop\Digital-Nurture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles>cd ProxyPatternExample

C:\Users\Hmbk\Desktop\Digital-Nurture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles\ProxyPatternExample>dotnet run

C:\Users\Hmbk\Desktop\Digital-Nurture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles\ProxyPatternExample\Program.cs(36,16): **warning CS8618:** Non-nullable field 'realImage' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the field as nullable.

First Display:

Loading sample.jpg from remote server...

Displaying sample.jpg

Second Display:

Displaying sample.jpg

main* 0 1 Projects: ProxyPattern Debug Any CPU C# Workspace Dev Mode Go Live

The screenshot displays the Visual Studio IDE with a C# file named `Program.cs` open. The code implements the Observer Pattern for a stock market application. It defines an `IObserver` interface with `Update()` and `Display()` methods. Two concrete classes, `MobileApp` and `WebApp`, implement these methods. The `StockMarket` class maintains a list of `IObserver` instances and provides methods to register observers, set prices, and notify them. The `Program` class contains the `Main` method, which creates a `StockMarket` instance, registers the `MobileApp` and `WebApp` observers, sets the initial price to 3500, and then updates it to 3600. The terminal output shows the execution of the program, displaying the price updates for both the mobile and web applications.

```
4 namespace ObserverPatternExample
70 class Program
72 static void Main(string[] args)
73 {
74     StockMarket stock = new StockMarket("CTS");
75
76     IObserver mobile = new MobileApp();
77     IObserver web = new WebApp();
78
79     stock.RegisterObserver(mobile);
80     stock.RegisterObserver(web);
81
82     stock.SetPrice(3500);
83     Console.WriteLine();
84
85     stock.SetPrice(3600);
86
87     Console.ReadKey();
88 }
89 }
```

Displaying sample.jpg

C:\Users\Hmbk\Desktop\Digital-Nurture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles\ProxyPatternExample>cd ..

C:\Users\Hmbk\Desktop\Digital-Nurture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles>cd ObserverPatternExample

C:\Users\Hmbk\Desktop\Digital-Nurture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles\ObserverPatternExample>dotnet run

Mobile App: CTS price updated to 3500
Web App: CTS price updated to 3500

Mobile App: CTS price updated to 3600
Web App: CTS price updated to 3600

The screenshot shows the Visual Studio IDE with a C# file named `Program.cs` open. The code defines a `StrategyPatternExample` namespace containing a `PaymentContext` class. The class has a private `IPaymentStrategy` field, a `SetStrategy` method, and an `ExecutePayment` method. The `ExecutePayment` method calls `paymentStrategy.Pay(amount)`. The output window shows the command `dotnet run` being executed, resulting in two lines of output: `Paid Rs.5000 using Credit Card` and `Paid Rs.3000 using PayPal`. A warning message is also displayed: `warning CS8618: Non-nullable field 'paymentStrategy' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the field as nullable.`

```
3 namespace StrategyPatternExample
24 }
25
26 2 references
27 class PaymentContext
28 {
29     2 references
30     private IPaymentStrategy paymentStrategy;
31
32     2 references
33     public void SetStrategy(IPaymentStrategy paymentStrategy)
34     {
35         this.paymentStrategy = paymentStrategy;
36     }
37
38     2 references
39     public void ExecutePayment(double amount)
40     {
41         paymentStrategy.Pay(amount);
42     }
43 }
```

^C
C:\Users\Hmbk\Desktop\Digital-Nuture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles\ObserverPatternExample>cd ..
C:\Users\Hmbk\Desktop\Digital-Nuture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles\StrategyPatternExample>dotnet run
C:\Users\Hmbk\Desktop\Digital-Nuture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles\StrategyPatternExample\Program.cs(28,34): warning CS8618: Non-nullable field 'paymentStrategy' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the field as nullable.
Paid Rs.5000 using Credit Card
Paid Rs.3000 using PayPal

The image shows a Visual Studio Code editor window with a C# file named `Program.cs` open. The file is part of a project named `CommandPatternExample`. The code implements the Command Pattern for a light control system. It defines a `Light` class, `LightOnCommand`, `LightOffCommand`, and a `RemoteControl` class. The `Program` class contains a `Main` method that demonstrates the usage of these classes.

```
65     }
66 }
67
68 class Program
69 {
70     static void Main(string[] args)
71     {
72         Light light = new Light();
73
74         ICommand lightOn = new LightOnCommand(light);
75         ICommand lightOff = new LightOffCommand(light);
76
77         RemoteControl remote = new RemoteControl();
78
79         remote.SetCommand(lightOn);
80         remote.PressButton();
81
82         remote.SetCommand(lightOff);
```

The terminal window at the bottom shows the execution of the program. It displays the current directory, runs `dotnet run`, and shows the output: "Light is ON" and "Light is OFF". A warning message (CS8618) is also visible, indicating a non-nullable field 'command' must contain a non-null value when exiting the constructor.

^C
C:\Users\Hmbk\Desktop\Digital-Nuture-DotNetFSE\Deepskilling\Engineering concepts\Design Pattern ns and Principles\StrategyPatternExample>cd ..

C:\Users\Hmbk\Desktop\Digital-Nuture-DotNetFSE\Deepskilling\Engineering concepts\Design Pattern ns and Principles>cd CommandPatternExample

C:\Users\Hmbk\Desktop\Digital-Nuture-DotNetFSE\Deepskilling\Engineering concepts\Design Pattern ns and Principles\CommandPatternExample>dotnet run
C:\Users\Hmbk\Desktop\Digital-Nuture-DotNetFSE\Deepskilling\Engineering concepts\Design Pattern ns and Principles\CommandPatternExample\Program.cs(55,26): warning CS8618: Non-nullable field 'command' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the field as nullable.
Light is ON
Light is OFF

VS Code Explorer: Digital-Nurture-DotNetFSE

Program.cs ...\DependencyInjectionExample U | Program.cs ...\MVCPatternExample U X

Engineering concepts > Design Patterns and Principles > MVCPatternExample > Program.cs > {} MVCPatternExample

3 namespace MVCPatternExample

64 }

65

66 0 references

66 class Program

67 {

68 0 references

68 static void Main(string[] args)

69 {

70 Student student = new Student("Hemambika", 109, "A");

71

72 StudentView view = new StudentView();

73

74 StudentController controller = new StudentController(student, view);

75

76 controller.UpdateView();

77

78 Console.WriteLine();

79

80 controller.SetStudentName("Hema");

81 controller.SetStudentGrade("A+");

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

dotnet

Name: Hema
ID: 101
Grade: A+
^C
C:\Users\Hmbk\Desktop\Digital-Nurture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles\MVCPatternExample>dotnet run
Student Details
Name: Hemambika
ID: 109
Grade: A

Student Details
Name: Hema
ID: 109
Grade: A+

main* | 0 0 | Projects: MVCPattern | Debug Any CPU | C# Workspace Dev Mode | Go Live

The screenshot displays the Visual Studio IDE with a C# project named 'DependencyInjectionExample'. The code in 'Program.cs' defines a 'CustomerService' class that uses 'ICustomerRepository' for data access. The terminal window shows the command 'dotnet run' being executed, which results in the output 'Customer ID: 101, Name: Hemambika'.

```
namespace DependencyInjectionExample
{
    class CustomerService
    {
        private ICustomerRepository repository;

        public CustomerService(ICustomerRepository repository)
        {
            this.repository = repository;
        }

        public void GetCustomer(int id)
        {
            Console.WriteLine(repository.FindCustomerById(id));
        }
    }
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

C:\Users\Hmbk\Desktop\Digital-Nurture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles>cd DependencyInjectionExample

C:\Users\Hmbk\Desktop\Digital-Nurture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles\DependencyInjectionExample>dot net

'dot' is not recognized as an internal or external command, operable program or batch file.

C:\Users\Hmbk\Desktop\Digital-Nurture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles\DependencyInjectionExample>dotnet run

Customer ID: 101, Name: Hemambika

^C

C:\Users\Hmbk\Desktop\Digital-Nurture-DotNetFSE\Deepskilling\Engineering concepts\Design Patterns and Principles\DependencyInjectionExample>dotnet run

Customer ID: 109, Name: Hemambika

main* 0 0 Projects: DependencyInjection Debug Any CPU C# Workspace Dev Mode Go Live